Understanding Two Machine Learning Programs

A line-by-line guide written for students with no Python background

Experiment 2 — Exploratory Data Analysis | Experiment 3 — Gradient Descent from Scratch

> **Read this first**
>
> **This guide assumes you have never written a line of Python.** Part 0 teaches you the eight ideas you need before any of the code makes sense. Read it once, slowly. After that, Parts 1 and 2 walk through both programs line by line, and every screenshot in this document is the actual output the programs produced — nothing is mocked up.

What is in this document

1. **Part 0 — Python survival kit.** Eight concepts: libraries, variables, DataFrames, arrays, functions, loops, f-strings, and indentation.
2. **Part 1 — Program 1 (experiment2.ipynb).** Exploratory Data Analysis on a 50,000-student placement dataset, explained cell by cell with all six output charts.
3. **Part 2 — Gradient descent by hand.** The same algorithm on three students and two weights, every number worked out on paper, plus the learning-rate experiments.
4. **Part 3 — Program 2 (Exp_3).** Batch and Mini-batch Gradient Descent written from scratch, compared against scikit-learn, with real console output and loss curves.
5. **Part 4 — Dataset download links.** Where to get the data, plus a script to generate an equivalent file if the original is unavailable.
6. **Part 5 — How to run both programs yourself** on Google Colab, step by step.
7. **Part 6 — Errors you will hit, and how to fix them.**
8. **Part 7 — Viva questions with model answers.**

Part 0 — The Python Survival Kit

Everything in both programs is built out of the eight ideas below. If you understand these, you can read the rest of this document without ever having programmed before.

1. A library is a toolbox someone else already built

You do not write mathematics or charting code yourself. You borrow it. The line `import numpy as np` means: "fetch the toolbox called NumPy, and let me refer to it by the short nickname np." From then on, `np.mean(...)` means "use the mean tool from the NumPy toolbox."

Library	Nickname	What it does
numpy	np	Fast mathematics on grids of numbers. This is what actually performs the gradient descent arithmetic.
pandas	pd	Loads spreadsheets and CSV files, and lets you filter, group, and clean them.
matplotlib.pyplot	plt	Draws charts — lines, bars, histograms.
seaborn	sns	Sits on top of matplotlib and makes prettier statistical charts with less code.
scikit-learn	sklearn	Ready-made machine learning models, used here as the correct answer to check our own code against.

2. A variable is a labelled box

The single `=` sign does not mean "equals" in the mathematical sense. It means "put the thing on the right into the box named on the left." So `df = pd.read_csv("data.csv")` means: read the file, then store the result in a box called `df`. Whenever you later write `df`, Python fetches whatever is in that box.

The double `==` sign is the one that asks a question: "are these two things the same?" It answers True or False. Confusing `=` with `==` is the single most common beginner mistake — and, as you will see in Part 1, a misuse of `==` is exactly what silently broke one chart in Program 1.

3. A DataFrame is a spreadsheet living inside the computer's memory

When pandas reads a CSV file, it produces a **DataFrame** — think of it as an Excel sheet you control with typed commands instead of a mouse. It has named columns and numbered rows. The variable is almost always called `df`, short for DataFrame.

You write	It means
`df.shape`	How many rows and columns? Answers as (rows, columns).
`df.head()`	Show me the first five rows.
`df['CGPA']`	Give me just the CGPA column.
`df[['CGPA', 'Projects']]`	Give me those two columns as a smaller table. Note the double brackets.
`df.isnull()`	For every single cell, is it empty? True or False.
`df.isnull().sum()`	Count how many empty cells there are in each column.
`df.groupby('Stream')`	Split the table into piles, one pile per Stream (CS, IT, ECE...).

4. A NumPy array is a pure grid of numbers

A DataFrame carries names, text, and labels. A NumPy array carries numbers and nothing else — which makes it much faster. `.values` strips a DataFrame down to a bare array. Its `shape` tells you its dimensions: `(50000, 12)` means 50,000 rows and 12 columns.

The symbol `@` means **matrix multiplication**. In plain terms: multiply every student's twelve numbers by twelve corresponding weights, add the results, and you get that student's predicted salary — all 40,000 students at once, in a single instruction. This is the reason the program runs in seconds instead of minutes.

5. A function is a recipe you can reuse

EXAMPLE

```
def compute_mse(X, y, theta):
    preds = X @ theta
    errors = preds - y
    return np.mean(errors ** 2)
```

The word `def` means "define a recipe." The name is `compute_mse`. The things in brackets — X, y, theta — are the ingredients you must hand it. `return` is what the recipe hands back to you. Writing the recipe does nothing on its own; it only runs when you *call* it, like `compute_mse(X_test_b, y_test, theta_gd)`.

6. A loop repeats work

EXAMPLE

```
for epoch in range(500):
    ...do something...
```

This says: do the indented block 500 times. Each time round, the box `epoch` holds the current count (0, then 1, then 2, and so on). In Program 2 the loop runs 500 times because we want to improve our predictions in 500 small steps.

7. Indentation is not decoration — it is the grammar

Most languages use curly braces to show what belongs inside what. Python uses **spaces**. Everything indented under a `for` or `def` line belongs to it. Shift a line left or right by accident and you change the meaning of the program, or break it entirely. Always use four spaces, and never mix tabs and spaces.

8. f-strings print numbers neatly

The line `print(f"Test RMSE = {rmse_gd:.4f}")` has three parts. The `f` before the quote says "this text contains slots." The `{rmse_gd}` is the slot — Python drops the value of that variable in. The `:.4f` is the formatting instruction: show it as a decimal number with exactly four digits after the point. Similarly `{...:<20}` pads text to 20 characters on the left and `{...:>12}` right-aligns it in 12 characters, which is how the neat summary table at the end of Program 2 is built.

Quick symbol reference

Symbol	Read it as
`#`	Everything after this on the line is a note for humans. Python ignores it.
`=`	Put the right-hand value into the left-hand box.
`==`	Ask: are these two equal? Gives True or False.
`@`	Matrix multiplication.
`**`	Raise to a power. errors ** 2 means each error squared.
`-=`	Subtract from the box and store the result back. theta -= x means theta = theta - x.
`.`	Reach inside. df.shape means "the shape belonging to df."
`a[2:5]`	Take items 2, 3 and 4. The start is included, the end is not. Counting begins at 0.

Part 1 — Program 1: Exploratory Data Analysis

File: `experiment2.ipynb` · **Platform:** Google Colab · **Cells:** 14

What this program is for

Before you build any model, you must look at your data. This program does exactly that and nothing more — it makes no predictions. It answers six questions about a dataset of 50,000 engineering students:

- How big is the dataset, and what is in it?
- Which values are missing, and are they missing in a worrying pattern?
- What shape does each numeric column have — is it a bell curve, or lopsided?
- Does placement rate differ between branches?
- Which features move together, and which are unrelated?
- Are there extreme outlier students who might distort a model later?

This activity is called **Exploratory Data Analysis (EDA)**. Finally, the program saves all five charts into a single PDF report and downloads it.

The dataset at a glance

50,000 rows and 32 columns. Each row is one student. The columns fall into five groups:

Group	Columns
Identity and background	StudentID, Gender, City, CollegeTier, Stream, Specialisation, Hostel, HistoryOfBacklogs
Academic record	SGPA_Sem1 through SGPA_Sem8, CGPA, AttendancePercent, CGPA_Tier
Experience and activity	Internships, Projects, Workshops, Certifications, Publications, ExtraCurricular
Assessment scores	AptitudeTestScore, SoftSkillsRating, CodingTestScore, MockInterviewScore
Outcomes	PlacementStatus (0 or 1), Salary Package (in lakhs per annum), IsAnomaly

Cell 0 — Upload the CSV file into Colab

CODE

```
from google.colab import files

uploaded = files.upload()
```

Google Colab runs on a Google server, not on your laptop, so it cannot see your files. This cell puts a **Choose Files** button on screen; you pick the CSV from your computer and it is copied up to the server. Colab confirms it:

OUTPUT

```
Saving placement_predict_50k Dataset.csv to placement_predict_50k Dataset.csv
```

> **Important if you are not using Colab**
> This cell works only inside Google Colab. If you are running the notebook in Jupyter or VS Code on your own machine, delete this cell and simply place the CSV in the same folder as the notebook.

Cell 1 — Load the toolboxes

CODE

```
import pandas as pd
import numpy as np
```

```
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.backends.backend_pdf import PdfPages

sns.set_style("whitegrid")   # optional: gives cleaner-looking charts
```

The first four lines are the standard data-science toolkit. `PdfPages` is the specific tool that will collect several charts into one PDF at the end. `sns.set_style("whitegrid")` is purely cosmetic — it puts a faint grid behind every chart so values are easier to read off.

This cell produces no output. Silence here means success.

Cell 2 — Read the file into a DataFrame

CODE

```
df = pd.read_csv("placement_predict_50k Dataset.csv")
```

One line does all the work: open the CSV, work out where each column starts and ends, guess whether each column holds numbers or text, and build the table in memory. Note the **space** between `50k` and `Dataset.csv`. Filenames must match character for character, including spaces and capitals, or you get a FileNotFoundError.

Cell 3 — Look at the first five rows

CODE

```
print("First 5 rows of the dataset:")
display(df.head())
```

Never trust a file you have not looked at. `df.head()` returns the first five rows. `display()` is a Colab convenience that renders it as a proper formatted table rather than plain text.

OUTPUT

```
First 5 rows of the dataset:

   StudentID  Gender       City CollegeTier Stream Specialisation Hostel  \
0          1    Male  Ahmedabad       Tier2    ECE     Networking     No
1          2  Female     Mumbai       Tier2    ECE    DataScience    Yes
2          3    Male    Kolkata       Tier2     IT    DataScience    Yes
3          4    Male     Jaipur       Tier1     CS             AI     No
4          5    Male       Pune       Tier2     IT    DataScience    Yes

   AptitudeTestScore  SoftSkillsRating  CodingTestScore  MockInterviewScore  \
0               66.7               2.2             49.4                47.8
1               48.2               2.4             26.7                25.8
2               73.8               2.8             67.7                41.5
3               69.8               2.7             66.9                48.0
4               73.1               2.1             71.7                61.7

   ExtraCurricular  CGPA_Tier  PlacementStatus  IsAnomaly  Salary Package
0                0        Low                0          0            0.00
1                0        Low                0          0            0.00
2                0        Low                1          0            3.89
3                0        Mid                1          0            8.37
4                1       High                1          0           18.99

[5 rows x 32 columns]
```

Two things to notice, because both matter later. First, the `...` in the middle means pandas hid some columns to fit the width — the data is all there. Second, and much more importantly: **PlacementStatus holds the numbers 0 and 1, not the words "Placed" and "Not Placed."** Students 1 and 2 show 0 with a salary of 0.00; students 3, 4 and 5 show 1 with real salaries. Remember this.

Cell 4 — Size, column names, data types, and missing values

CODE

```
print("\nShape of dataset (rows, columns):", df.shape)
```

```
print("\nColumn names:")
print(df.columns.tolist())

print("\nData types and non-null counts:")
df.info()

print("\nMissing values per column:")
print(df.isnull().sum())
```

Four checks in one cell. `\n` inside a printed string means "start a new line here" — it is only there to space the output out. `df.info()` is the most useful single command in pandas: it lists every column with its data type and how many non-empty values it has.

OUTPUT

```
Shape of dataset (rows, columns): (50000, 32)

Data types and non-null counts:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 32 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   StudentID           50000 non-null  int64
 1   Gender              50000 non-null  object
...
 20  Workshops           45512 non-null  float64
 23  AptitudeTestScore   45971 non-null  float64
 24  SoftSkillsRating    46474 non-null  float64
 25  CodingTestScore     47024 non-null  float64
 26  MockInterviewScore  45043 non-null  float64
 29  PlacementStatus     50000 non-null  int64
 31  Salary Package      50000 non-null  float64
dtypes: float64(16), int64(8), object(8)
memory usage: 12.2+ MB

Missing values per column:
Workshops             4488
AptitudeTestScore     4029
SoftSkillsRating      3526
CodingTestScore       2976
MockInterviewScore    4957
(all other columns: 0)
```

How to read this output

- **50000 rows, 32 columns.** A comfortable size — big enough to be realistic, small enough to run in seconds.
- `int64` means whole numbers, `float64` means decimals, and `object` means text. Note that **PlacementStatus is int64** — this is the confirmation of what we spotted in Cell 3.
- **Five columns have gaps.** Workshops is missing 4,488 values, MockInterviewScore 4,957, and so on — roughly 6% to 10% of each. Every other column is complete.
- Why the gaps matter: most machine learning algorithms crash on empty cells. Program 2 deals with this by filling each gap with that column's median.

Cell 5 — Creating a placement flag (and the bug hiding in it)

CODE

```
df['Placed_Flag'] = df['PlacementStatus'].apply(lambda x: 1 if x == 'Placed' else 0)
```

The intention is reasonable: create a new column of 1s and 0s so we can average it to get a placement percentage. `.apply()` runs a small rule on every row, and `lambda x: 1 if x == 'Placed' else 0` is that rule written inline — "given a value x, give back 1 if x is the text Placed, otherwise give back 0."

> **This cell contains a real bug — and it is the most valuable thing in this program**

The rule compares each value against the **text 'Placed'**. But we established in Cells 3 and 4 that PlacementStatus contains the **numbers 0 and 1**. In Python, the number 1 is not equal to the text "Placed" — so the test is *never* true, the else branch always fires, and every single row gets 0.

Python raises no error. The cell runs, produces no output, and looks perfectly fine. The damage only surfaces two cells later, when the placement-rate chart comes out flat at zero.

The fix is one line:

```
df['Placed_Flag'] = df['PlacementStatus']
```

Or, if you want it to work whether the column holds numbers or text:

```
df['Placed_Flag'] = df['PlacementStatus'].apply(lambda x: 1 if x in [1, 'Placed'] else 0)
```

The lesson worth writing in your record: a program that runs without an error message is not the same as a program that is correct. Always sanity-check your numbers against what you know about the data.

Cell 6 — Choose the columns to chart

CODE

```
num_cols = ['CGPA', 'AttendancePercent', 'AptitudeTestScore',
            'CodingTestScore', 'SoftSkillsRating', 'Salary Package']
```

A **list** is an ordered collection written inside square brackets. There are 24 numeric columns in the dataset, but charting all of them would be unreadable, so six representative ones are picked. Six is chosen deliberately: it fits a tidy 2-row by 3-column grid of charts.

Cell 7 — Missingness heatmap

CODE

```
fig1 = plt.figure(figsize=(10, 6))
sns.heatmap(df.isnull(), cbar=False, cmap="viridis", yticklabels=False)
plt.title("Missingness Heatmap - Placement Prediction Dataset")
plt.xlabel("Columns")
plt.tight_layout()
plt.show()
```

- `plt.figure(figsize=(10, 6))` creates a blank canvas 10 inches wide and 6 inches tall, and stores it in `fig1` so it can be added to the PDF later.
- `df.isnull()` turns the whole table into True/False — True wherever a cell is empty.
- `sns.heatmap(...)` paints that True/False grid as an image: one thin horizontal line per student, one vertical strip per column. `cbar=False` hides the colour scale (pointless for True/False) and `yticklabels=False` hides the row numbers, because 50,000 labels would be a solid black smear.
- `plt.tight_layout()` stops the axis labels being cut off; `plt.show()` displays it.

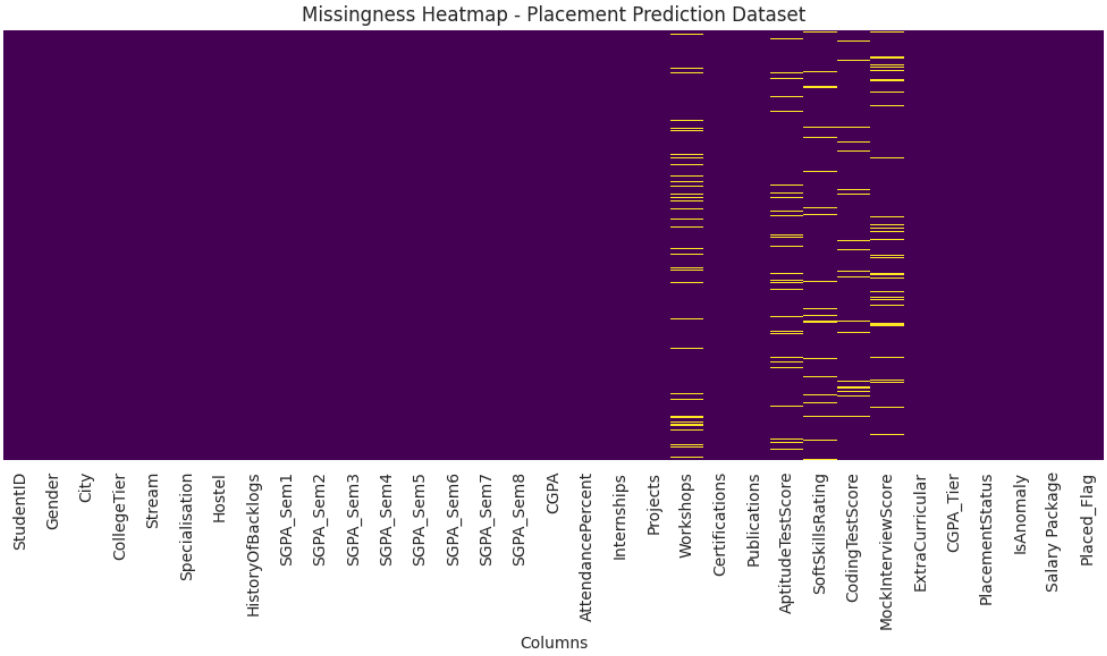

Figure 1: Missingness heatmap. Each vertical band is one column; the speckled bands are the five columns with gaps.

How to read it: most of the image is one solid colour — those are the complete columns. Five bands are flecked with a different colour, and those flecks are the missing values. Crucially, the flecks are scattered evenly from top to bottom rather than clustered in a block. That tells you the data is missing **at random**, not because of some systematic failure such as one whole college never reporting scores. Random gaps are safe to fill in with a median; systematic gaps are not.

Cell 8 — Six histograms in one figure

CODE

```
fig2, axes = plt.subplots(2, 3, figsize=(15, 8))

for i, col in enumerate(num_cols):
    row, colpos = divmod(i, 3)
    sns.histplot(df[col].dropna(), bins=30, kde=True,
                 ax=axes[row, colpos], color='steelblue')
    axes[row, colpos].set_title(f'Distribution of {col}')

plt.tight_layout()
plt.show()
```

A **histogram** answers "how are these values spread out?" It chops the range into 30 buckets and draws a bar showing how many students fall in each.

- `plt.subplots(2, 3, ...)` makes one canvas divided into a 2×3 grid of six mini-charts. `axes` is the grid; `axes[0, 2]` is the top-right slot.
- `enumerate(num_cols)` walks the list and hands you both the position and the name each time: (0, "CGPA"), then (1, "AttendancePercent"), and so on.
- `divmod(i, 3)` converts a position 0–5 into a row and column. For i = 4 it gives (1, 1) — second row, second column. This is the neat trick that fills the grid without writing six near-identical blocks of code.
- `.dropna()` skips empty cells for this chart only; `kde=True` overlays a smooth curve showing the underlying shape.

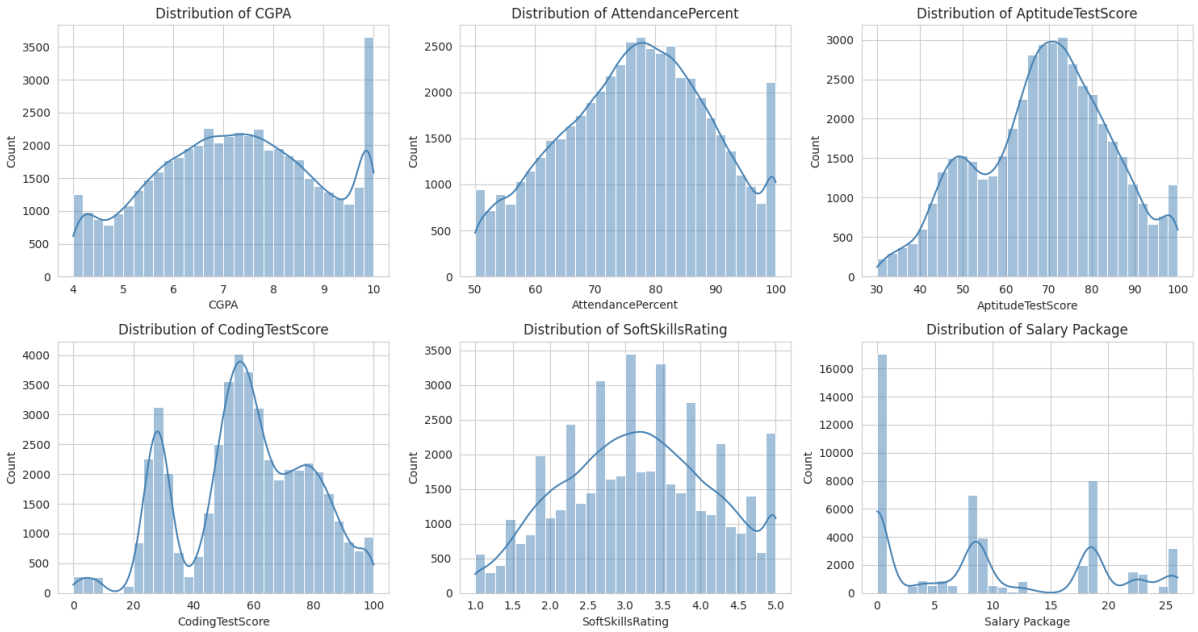

Figure 2: Distribution of six key numeric columns. Note the spike at zero in Salary Package.

What each panel is telling you

- **CGPA and AptitudeTestScore** are roughly bell-shaped — most students cluster near the middle with few at the extremes. This is normal and healthy.
- **Salary Package is the interesting one.** It has a tall spike at exactly zero and then a spread of real values. That spike is not an error: it is every unplaced student, recorded as earning nothing. This single observation should shape your entire modelling strategy — a regression trained on this column is really learning two different things at once.
- **SoftSkillsRating** sits on a narrow 1–5 scale, so its histogram looks blockier than the others.

Cell 9 — Placement rate by stream

CODE

```
fig3 = plt.figure(figsize=(10, 6))

placement_rate = (
    df.groupby('Stream')['Placed_Flag'].mean().sort_values(ascending=False) * 100
)

sns.barplot(x=placement_rate.index, y=placement_rate.values, palette='crest')
plt.title('Placement Rate (%) by Stream')
...
print(placement_rate.round(2))
```

Read the long line from the inside out: `groupby('Stream')` sorts students into six piles by branch; `['Placed_Flag']` picks the 1/0 column; `.mean()` averages it within each pile. The average of a column of 1s and 0s *is* the proportion — that is the whole reason for making the flag. `* 100` converts it to a percentage, and `sort_values` orders the bars highest first.

OUTPUT

```
Placement rate (%) by stream:
Stream
CS            0.0
Civil         0.0
ECE           0.0
EE            0.0
IT            0.0
Mechanical    0.0
Name: Placed_Flag, dtype: float64
```

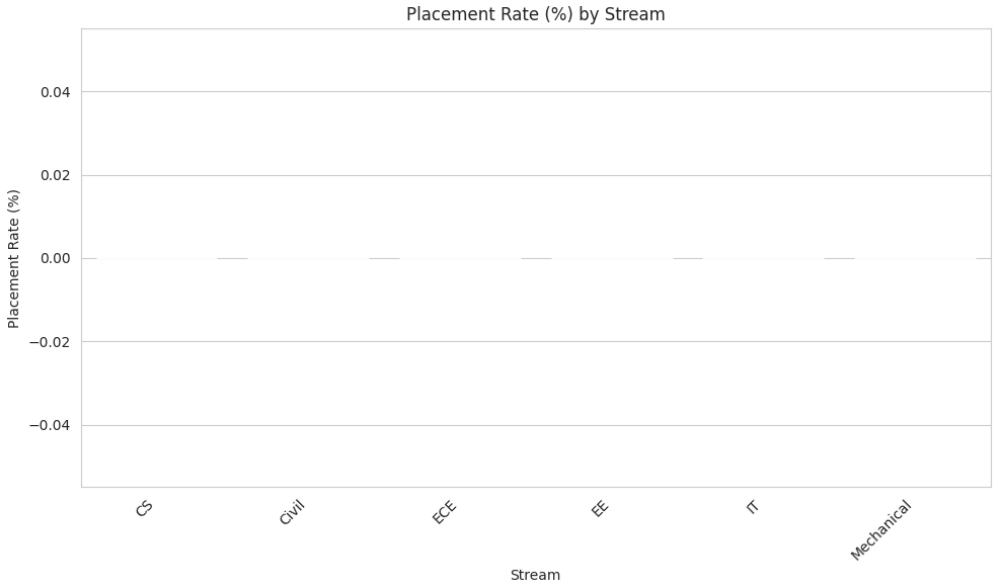

Figure 3: The chart as actually produced — every bar is zero, because of the Cell 5 bug.

This is the bug from Cell 5 surfacing

A 0.0% placement rate in every single branch is impossible. If it were true, no student in a 50,000-row dataset ever got a job — yet Cell 3 showed us students with salaries of 8.37 and 18.99 lakhs. The chart is not reporting reality; it is reporting that Placed_Flag is entirely zeros.

Always apply this test to your own output: is this number physically possible? An exact 0.0 or an exact 100.0 across every category is almost always a code fault, not a discovery.

After applying the one-line fix from Cell 5 and re-running, the chart becomes meaningful. Below is the corrected version. (This particular figure was produced on a stand-in dataset with the same structure, since the original file was not available to re-run — your exact percentages will differ, but the shape of the corrected output will look like this.)

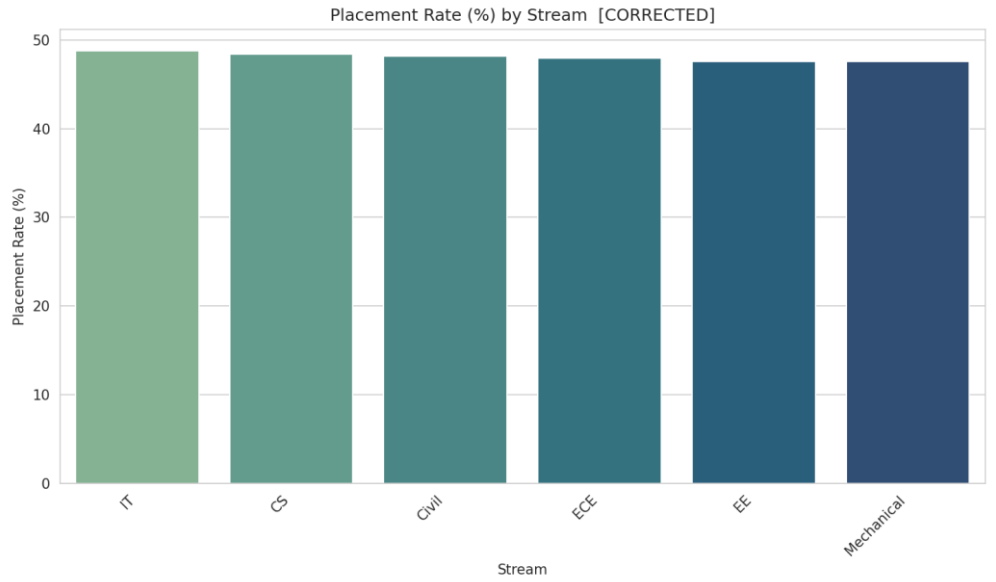

Figure 4: The same chart after fixing Placed_Flag — bars now show real, differing placement rates.

The warning message this cell also prints

OUTPUT

```
FutureWarning:
Passing `palette` without assigning `hue` is deprecated and will be
```

```
removed in v0.14.0. Assign the `x` variable to `hue` and set
legend=False for the same effect.
```

A **warning** is not an error. Your code still ran. Seaborn is telling you that this way of colouring bars will stop working in a future version. To silence it, change the barplot line to:

FIXED LINE

```
sns.barplot(x=placement_rate.index, y=placement_rate.values,
            hue=placement_rate.index, palette='crest', legend=False)
```

Cell 10 — Correlation heatmap

CODE

```
fig4 = plt.figure(figsize=(10, 8))

numeric_df = df.select_dtypes(include=['int64', 'float64'])
corr_matrix = numeric_df.corr()

sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='coolwarm',
            square=True, linewidths=0.5)
plt.title('Correlation Heatmap - Numerical Features')
plt.show()
```

A **correlation** is a single number between −1 and +1 describing how two columns move together. +1 means they rise in perfect lockstep, −1 means one rises exactly as the other falls, and 0 means they are unrelated. `.corr()` computes this for every possible pair at once.

- `select_dtypes(...)` keeps only numeric columns — you cannot correlate the text "Male" with anything.
- `annot=True` prints the actual number inside each square and `fmt='.2f'` shows it to two decimals.
- `cmap='coolwarm'` is the colour scheme: red for positive, blue for negative, pale for near zero. This is why you can spot patterns without reading a single digit.

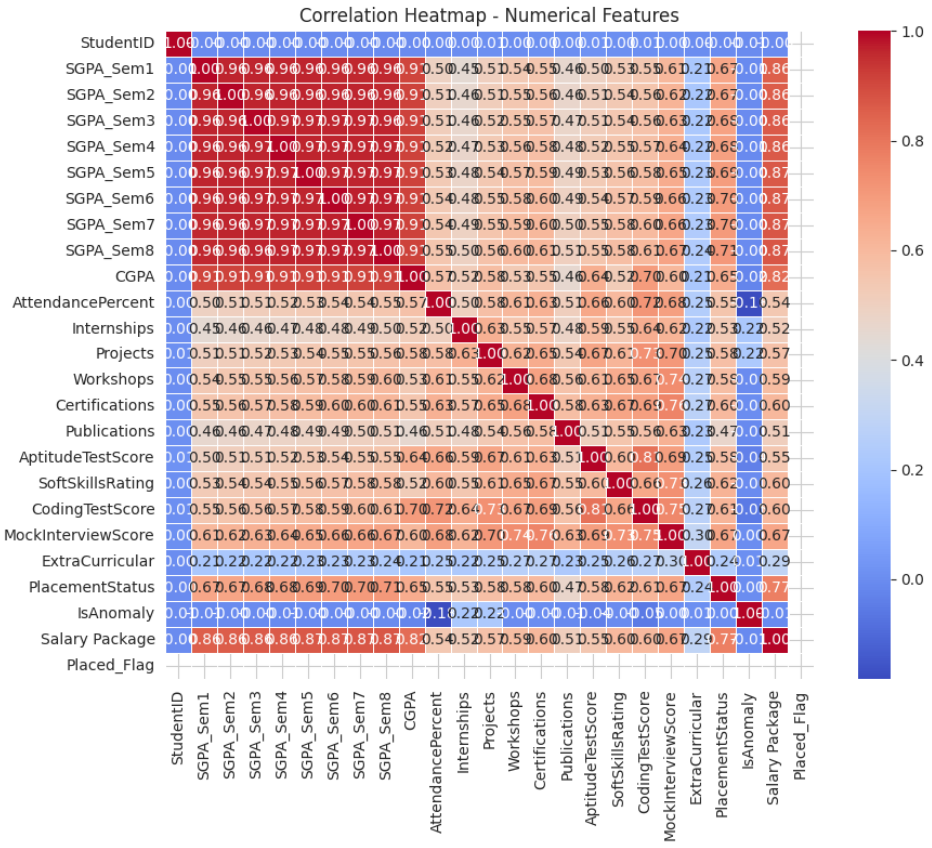

Figure 5: Correlation heatmap of all numeric columns. The red diagonal is every column against itself.

How to read it

- The dark red line running corner to corner is every column correlated with itself, which is always exactly 1.00. Ignore it.
- The eight SGPA columns form a solid red block with each other and with CGPA — unsurprising, since CGPA is computed from them. This is called multicollinearity, and it means you should not feed all nine into a model together.
- Scan the **Salary Package** row. Columns with strong red there are your genuinely predictive features. Columns that are pale are close to useless for prediction.
- **StudentID** should be near zero against everything. If it is not, something is wrong with how the file was generated.

Cell 11 — Boxplots for outlier detection

CODE

```
fig5, axes5 = plt.subplots(2, 3, figsize=(15, 8))

for i, col in enumerate(num_cols):
    row, colpos = divmod(i, 3)
    sns.boxplot(y=df[col], ax=axes5[row, colpos], color='lightcoral')
    axes5[row, colpos].set_title(f'Boxplot of {col}')

plt.tight_layout()
plt.show()
```

Same grid structure as Cell 8, different chart type. A boxplot compresses a whole column into five numbers:

Part of the box	Meaning
The line inside the box	The median — half the students are above it, half below.
The box itself	The middle 50% of students.
The whiskers	The normal range, reaching out to 1.5 times the box height.
Individual dots beyond	Outliers — unusually high or low students.

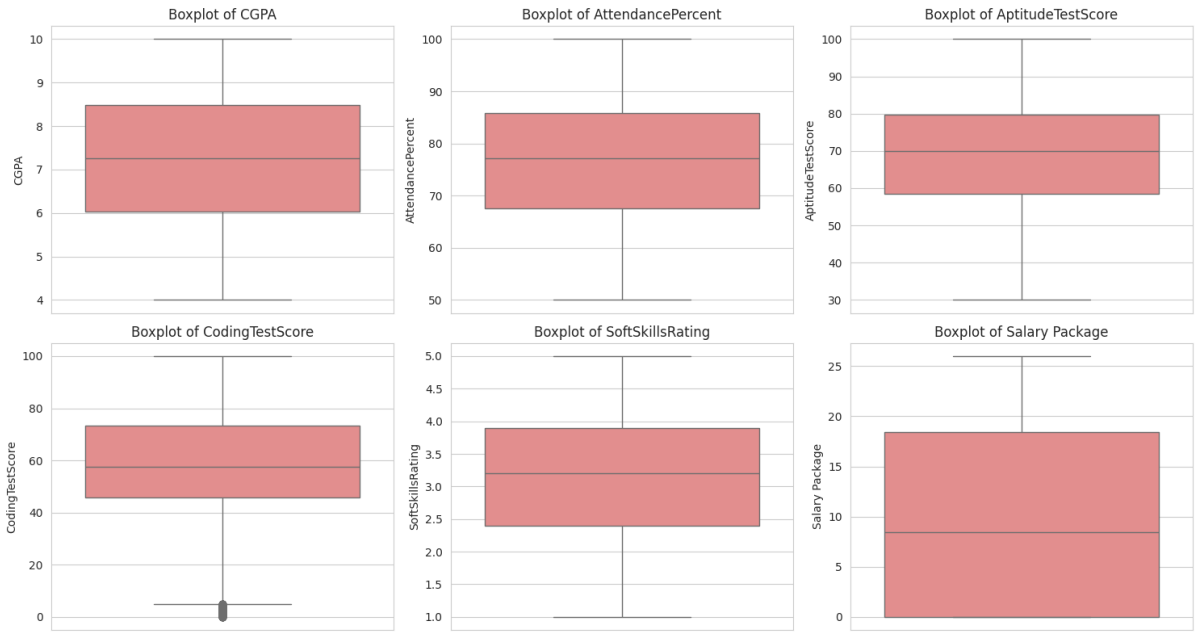

Figure 6: Boxplots of the same six columns. Dots beyond the whiskers are outliers.

Why you care: outliers can drag a regression line badly off course, because squared error punishes large mistakes disproportionately. Before modelling you must decide, for each column, whether an outlier is a data-entry error

(remove it) or a genuinely exceptional student (keep it). A student with a 45 LPA package is probably real; a student with a CGPA of 47 is not.

Cells 12 and 13 — Save all charts to a PDF and download it

CODE

```
pdf = PdfPages('EDA_Report.pdf')

pdf.savefig(fig1)   # Page 1: Missingness Heatmap
pdf.savefig(fig2)   # Page 2: Six Histograms
pdf.savefig(fig3)   # Page 3: Placement Rate by Stream
pdf.savefig(fig4)   # Page 4: Correlation Heatmap
pdf.savefig(fig5)   # Page 5: Boxplots

pdf.close()
print("\nEDA_Report.pdf created successfully with 5 pages!")

files.download('EDA_Report.pdf')
```

This is why every figure was stored in a variable (`fig1` … `fig5`) back when it was created. `PdfPages` opens an empty PDF, each `savefig` adds one chart as one page, and `pdf.close()` finalises the file. **Forgetting pdf.close() produces a corrupt PDF that will not open** — the file is only properly written when it is closed. The last cell pushes the finished PDF from the Colab server back down to your computer.

Program 1 in one paragraph, for your record book

The program loads a 50,000-row, 32-column student placement dataset, inspects its structure and missing values, and produces five diagnostic visualisations: a missingness heatmap, six distribution histograms, a placement-rate bar chart by branch, a correlation heatmap, and six outlier boxplots. All five are exported to a single five-page PDF report. The analysis reveals that five assessment columns have 6–10% missing values distributed at random, that Salary Package has a large spike at zero representing unplaced students, and that the eight semester SGPA columns are highly collinear with CGPA. The placement-rate chart returned zero for every branch, which was traced to a type-comparison fault in the Placed_Flag construction rather than to the data itself.

Part 2 — Gradient Descent by Hand

Read this part before Part 3. Part 3 runs gradient descent on 40,000 students and 13 weights at once, which is impossible to follow with your eyes. So here we shrink the problem until it fits on paper: **three students, one input, two weights**. Every number below has been computed and checked. Work through it with a calculator once and the code in Part 3 becomes obvious.

The tiny dataset

Three students. We record how many hours each studied, and what they scored on a quiz out of 10.

Hours studied (x)	Marks obtained (y)
1	3
2	5
3	7

You can see the rule with your eyes: `marks = 2 × hours + 1`. So the correct answer is **w = 2 and b = 1**. That is exactly the point. We already know the destination, so we can watch gradient descent grope its way there from a standing start and check that it arrives.

The three ingredients

1. The model — a guess with two dials

CODE

```
prediction = w * x + b
```

`w` (the **weight**) controls how steep the line is. `b` (the **bias** or intercept) controls how high up it sits. Training means nothing more than turning these two dials until the line passes as close as possible to all three dots.

We start at **w = 0, b = 0**. This is a deliberately terrible starting guess — it predicts that every student scores zero, no matter how much they study.

2. The loss — one number for how wrong we are

MEAN SQUARED ERROR

```
MSE = (1/m) * sum of (prediction - actual) squared
```

For each student, take prediction minus actual. Square it — this makes every value positive, so overshooting by 2 and undershooting by 2 count the same, and it punishes big mistakes far more than small ones. Then average across all m students. **One number. Smaller is better. Zero is perfect.**

3. The gradient — which way is downhill

Calculus gives us the slope of the loss with respect to each dial. You do not need to derive these; you need to be able to read them:

CODE

```
gradient for w  =  (2/m) * sum of (error * x)
gradient for b  =  (2/m) * sum of (error)
```

Read the first one in words: *for each student, multiply their error by their input, add it all up, and average it.* If students with large x values tend to have large positive errors, this sum is large and positive, which tells us w is too big. The second is the same idea without the x, because b affects every student equally.

The update rule — the one line that is gradient descent

CODE

```
w = w - learning_rate * gradient_w
b = b - learning_rate * gradient_b
```

> **Why minus, not plus?**
> The gradient points **uphill** — in the direction that makes the loss *larger*. We want to go down, so we move in the opposite direction. That minus sign is doing all the work in the entire algorithm. Change it to a plus and the model will climb steadily towards maximum error.

Iteration 1 — worked in full

Starting values: `w = 0`, `b = 0`, learning rate `α = 0.1`, m = 3 students.

Step A — predict

Every prediction is `0 × x + 0 = 0`.

Student	x	Prediction (w·x + b)	Actual y
1	1	0 × 1 + 0 = 0	3
2	2	0 × 2 + 0 = 0	5
3	3	0 × 3 + 0 = 0	7

Step B — find the errors (prediction minus actual)

CODE

```
error 1 = 0 - 3 = -3
error 2 = 0 - 5 = -5
error 3 = 0 - 7 = -7
```

All three errors are negative, which means we are under-predicting every student. Sensible — our line is flat on the floor.

Step C — measure the loss

CODE

```
MSE = ( (-3)^2 + (-5)^2 + (-7)^2 ) / 3
    = ( 9 + 25 + 49 ) / 3
    = 83 / 3
    = 27.6667
```

Step D — compute the gradients

CODE

```
gradient_w = (2/3) * [ (-3)(1) + (-5)(2) + (-7)(3) ]
           = (2/3) * [ -3 - 10 - 21 ]
           = (2/3) * (-34)
           = -22.6667

gradient_b = (2/3) * [ (-3) + (-5) + (-7) ]
           = (2/3) * (-15)
           = -10.0000
```

Step E — update the dials

Both gradients are negative, so subtracting a negative increases both values. The algorithm has correctly worked out that the line needs to rise.

CODE

```
w = 0 - 0.1 * (-22.6667) = 0 + 2.26667 = 2.26667
b = 0 - 0.1 * (-10.0000) = 0 + 1.00000 = 1.00000
```

Look at what just happened

In **one step**, starting from a completely blind guess of (0, 0), we landed at (2.267, 1.000). The true answer is (2, 1). The loss fell from **27.6667 to 0.3319** — a drop of 98.8%.

This is why gradient descent works. It never searches randomly and it never tries every combination. It reads the slope and walks straight downhill.

Iteration 2 — the correction

Now `w = 2.26667` and `b = 1.0`.

Student	Prediction	Actual	Error
1	2.26667(1) + 1 = 3.2667	3	+0.2667
2	2.26667(2) + 1 = 5.5333	5	+0.5333
3	2.26667(3) + 1 = 7.8000	7	+0.8000

The errors are now **positive**. We have gone from under-predicting everybody to over-predicting everybody — the algorithm stepped slightly past the bottom of the valley. This sign flip is completely normal and is how gradient descent homes in: it overshoots, notices, and steps back a little less far.

CODE

```
MSE = (0.2667^2 + 0.5333^2 + 0.8^2) / 3 = 0.99556 / 3 = 0.33185

gradient_w = (2/3) * [0.2667(1) + 0.5333(2) + 0.8(3)]
           = (2/3) * 3.73333 = +2.48889
gradient_b = (2/3) * [0.2667 + 0.5333 + 0.8]
           = (2/3) * 1.6 = +1.06667

w = 2.26667 - 0.1 * 2.48889 = 2.01778
b = 1.00000 - 0.1 * 1.06667 = 0.89333
```

New MSE: **0.005267**. We are now extremely close to the true line.

The first five iterations at a glance

Iteration	w	b	MSE
0 (start)	0.00000	0.00000	27.666667
1	2.26667	1.00000	0.331852
2	2.01778	0.89333	0.005267
3	2.04385	0.90756	0.001304
4	2.03990	0.90850	0.001198
5	2.03926	0.91084	0.001141
Target	2.00000	1.00000	0.000000

Notice the pattern that every loss curve shows: **enormous progress early, then progressively smaller refinements.** Iteration 1 removes 98.8% of the error. Iterations 3 to 5 barely move the needle. After 1,000 iterations w and b reach 2.000000 and 1.000000 exactly.

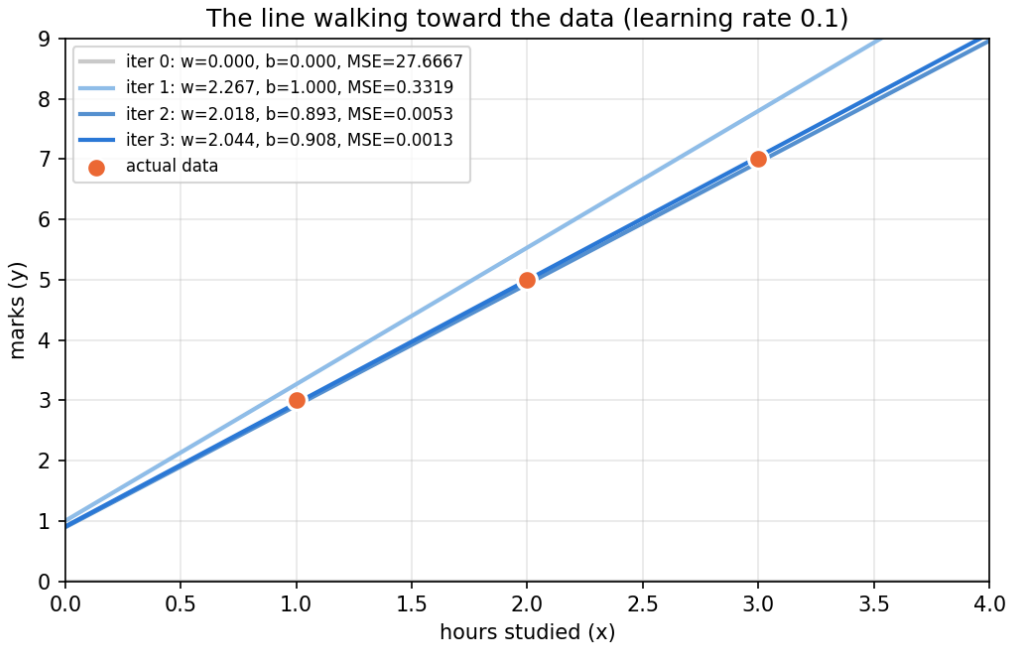

Figure 7: The line walking towards the data. Grey is the starting guess (flat on the floor); each darker blue line is one iteration later.

The learning rate — the one knob you must tune

The learning rate α decides how big each step is. It is the difference between arriving, crawling, and exploding. Here is the **same problem**, run 25 times, with nothing changed but α:

Learning rate	w after 25 steps	b after 25 steps	MSE	Verdict
0.001	0.4972	0.2195	15.84	Far too slow — barely moved
0.01	1.9336	0.8576	0.0787	Slow but working
0.10	2.0241	0.9452	0.00043	Good
0.15	2.0011	0.9974	0.0000009	Excellent
0.18	—	—	14.4	Unstable — oscillating
0.20	huge	huge	4.1 × 10^18	Diverged
0.45	infinite	infinite	overflow	Exploded completely

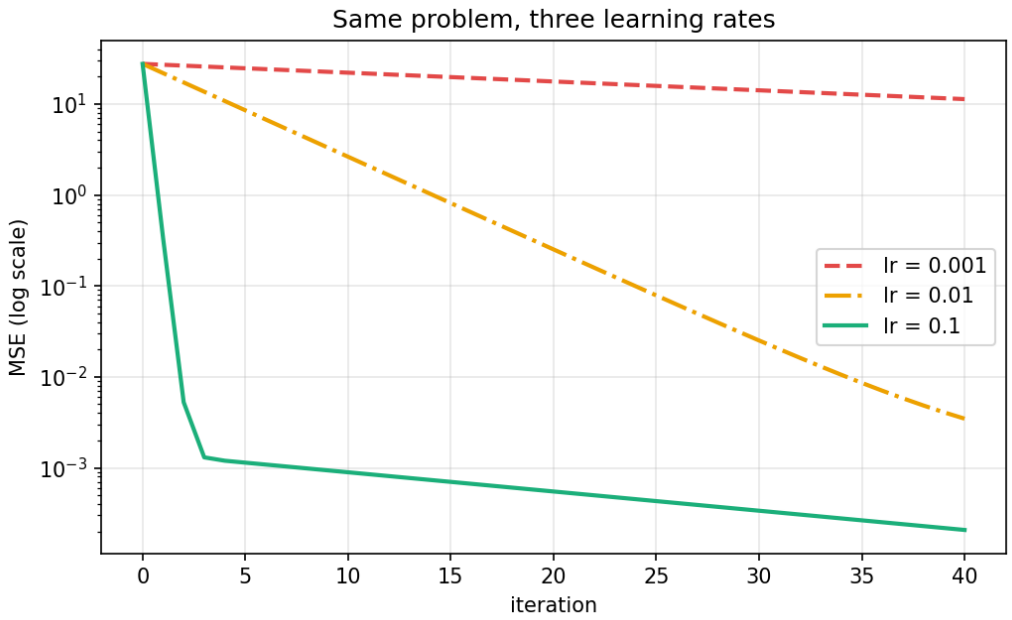

Figure 8: Three learning rates on the same problem. The vertical axis is logarithmic so all three fit on one chart.

What goes wrong at each extreme

- **Too small (0.001).** Each step is a shuffle. The loss does go down, but it would take tens of thousands of iterations to arrive. In a real project this looks like "my model isn't learning."
- **Too large (0.20 and above).** Each step leaps clean over the bottom of the valley and lands higher up the opposite side. The next gradient is therefore bigger, so the next leap is bigger still. Within a few iterations the numbers exceed what the computer can represent and you see `nan` or `inf`.

Figure 9: Divergence at learning rate 0.45. The loss climbs by a factor of roughly 20 every iteration.

There is no formula for the learning rate

You cannot calculate it. Every practitioner does the same thing: try 0.001, 0.01 and 0.1, plot the loss curve, and pick the largest value that still descends smoothly. This is why plotting the loss curve is not optional decoration — it is your only diagnostic instrument.

A useful rule of thumb: if the curve rises or shows nan, divide the learning rate by 10. If the curve is still falling steeply when the epochs run out, multiply it by 10.

Do it yourself — a lab exercise

Run the toy problem with each learning rate below and fill in the table. This makes an excellent addition to your lab record because it demonstrates the concept far better than a single successful run.

Learning rate	MSE after 25 steps	Converged? (Y/N)	Observation
0.001			
0.01			
0.10			
0.20			
0.45			

PASTE THIS INTO COLAB

```
import numpy as np

x = np.array([1., 2., 3.])
y = np.array([3., 5., 7.])

def run(lr, n_iter=25, show=5):
    w = b = 0.0
    for i in range(n_iter):
        preds = w * x + b
        errors = preds - y
        mse = np.mean(errors ** 2)
        grad_w = (2 / 3) * np.sum(errors * x)
        grad_b = (2 / 3) * np.sum(errors)
        if i < show:
            print(f"iter {i}: w={w:.5f} b={b:.5f} MSE={mse:.6f} "
                  f"grad_w={grad_w:.5f} grad_b={grad_b:.5f}")
        w -= lr * grad_w
        b -= lr * grad_b
    print(f"FINAL  lr={lr}: w={w:.5f} b={b:.5f} "
          f"MSE={np.mean((w * x + b - y) ** 2):.6g}")

for rate in [0.001, 0.01, 0.1, 0.2, 0.45]:
    run(rate)
    print()
```

From three students to forty thousand

Nothing conceptual changes when you scale up. Only the bookkeeping does. This table is the bridge to Part 3:

This toy example	Program 2 in Part 3
3 students	40,000 students
1 input (hours studied)	12 inputs (CGPA, coding score, internships, …)
2 dials: w and b	13 dials, stored together in one array called theta
`w * x + b`	`X @ theta` — one matrix multiply covers all students and all weights
`sum of (error * x)`	`X.T @ errors` — the same sum, done for all 13 weights at once
b tracked separately	A column of 1s glued on, so b becomes just another weight
About 25 iterations	500 epochs
Numbers checked by hand	Checked against scikit-learn instead

The single line `theta -= lr * gradient` in Part 3 is doing precisely what `w = w - 0.1 * (-22.6667)` did in Step E above. If you understood that step, you understand the algorithm. Everything else is NumPy handling the arithmetic in bulk.

Two misconceptions to avoid

- **"The gradient tells us which way to go."** Not quite — it points *uphill*. The minus sign in the update rule is what turns it into a descent. This is a common viva question.
- **"More epochs is always better."** Once the loss curve flattens, further epochs achieve nothing at all — you are taking steps on level ground. In our toy example, iterations 3 onwards changed the loss by less than 0.0002. Knowing when to stop is why you plot the curve.

Part 3 — Program 2: Gradient Descent from Scratch

File: `Exp_3` · **Goal:** implement the learning algorithm by hand and prove it matches a professional library

What this program is for

Program 1 only looked at data. This program actually *learns* from it. It predicts a student's salary package from twelve of their attributes, and it does so three separate ways:

9. **Batch Gradient Descent** — written from scratch, using nothing but arithmetic.
10. **Mini-batch Gradient Descent** — also from scratch, a faster variant.
11. **scikit-learn's LinearRegression** — the professional library, used as the known-correct answer.

If all three give the same result, your hand-written mathematics is provably correct. **That agreement is the entire point of the experiment** — not the accuracy of the prediction.

The idea, before any code

Part 2 worked this through on three students with a calculator. This is a compressed recap — if any of it feels shaky, go back to Part 2 rather than pushing on.

Step 1: What is linear regression?

Suppose salary depended on CGPA alone. You would plot every student as a dot and draw the straight line that passes closest to all of them: `salary = w × CGPA + b`. Here w controls the steepness and b is where the line starts.

We have twelve attributes rather than one, so the line becomes:

THE MODEL

```
salary = w1×CGPA + w2×Attendance + w3×Internships + ... + w12×ExtraCurricular + b
```

Those thirteen numbers (twelve weights plus b) are collectively called **theta**, written θ. Training the model means nothing more than finding the thirteen values of theta that make the predictions closest to the truth.

Step 2: What does "closest" mean? — the loss function

For each student, compute (predicted salary − actual salary). Square it, so that overshooting by 2 and undershooting by 2 are penalised the same and so that big mistakes hurt disproportionately. Average across all students. That average is the **Mean Squared Error (MSE)**, and it is the single number we are trying to make as small as possible.

MSE is in squared units, which are meaningless (rupees-squared). Taking the square root gives **RMSE**, which is back in lakhs per annum and can be read directly: "on average, our prediction is off by this much."

Step 3: What is gradient descent?

Picture yourself standing on a foggy hillside, trying to reach the valley floor. You cannot see the bottom, but you can feel which way the ground slopes under your feet. So you take a small step downhill, feel again, step again. Repeat enough times and you arrive at the bottom.

In the analogy	In the program
The hillside	The loss function — every possible setting of theta has a height (its MSE).
Your position	The current values of theta.
The slope under your feet	The gradient — calculus tells us the direction of steepest increase.
Step size	The learning rate (0.05 here). Too small and it takes forever; too large and you overshoot the valley entirely.

In the analogy	In the program
One step	One update: theta = theta − learning_rate × gradient.
Number of steps taken	Epochs (500 here).

The minus sign is the whole trick: the gradient points *uphill*, so we move in the opposite direction to go down.

Step 1 of the code — Load and prepare the data

CODE

```
df = pd.read_csv("placement_predict_50k_Dataset.csv")

FEATURES = [
    "CGPA", "AttendancePercent", "Internships", "Projects",
    "Workshops", "Certifications", "Publications",
    "AptitudeTestScore", "SoftSkillsRating",
    "CodingTestScore", "MockInterviewScore", "ExtraCurricular",
]
TARGET = "Salary Package"

data = df[FEATURES + [TARGET]].copy()
data[FEATURES] = data[FEATURES].fillna(data[FEATURES].median())

X = data[FEATURES].values
y = data[TARGET].values.reshape(-1, 1)
```

- `FEATURES` lists the twelve inputs. Text columns such as Gender and City are excluded because the from-scratch mathematics only handles numbers. StudentID, PlacementStatus and IsAnomaly are also excluded — and PlacementStatus is excluded for a deeper reason: knowing whether a student was placed would give away almost all of the answer. Including it is called **data leakage**, and it makes a model look brilliant in testing and useless in reality.
- `.copy()` takes a genuine duplicate rather than a reference, so that editing `data` cannot accidentally alter `df`.
- `.fillna(...median())` plugs every gap found in Program 1 with the middle value of that same column. The median is preferred over the mean because it is not dragged around by outliers.
- `X` becomes a numeric grid of shape (50000, 12) and `y` a single column of shape (50000, 1). `.reshape(-1, 1)` forces the target into an upright column; the −1 means "work out that dimension for me." Without this the matrix multiplication later would silently compute the wrong thing.

Step 2 — Split, scale, and add the bias column

CODE

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s  = scaler.transform(X_test)

n_samples, n_features = X_train_s.shape
X_train_b = np.hstack([np.ones((n_samples, 1)), X_train_s])
X_test_b  = np.hstack([np.ones((X_test_s.shape[0], 1)), X_test_s])
```

Why split the data?

`test_size=0.2` means the model is trained on 40,000 students (80%) and then judged on the 10,000 students (20%) it has never seen. Testing on the same data you trained on is like setting an exam from the answer key you already handed out — the score tells you nothing about real ability. `random_state=42` fixes the random shuffle so you get the same split every run, which is essential for reproducibility. The number 42 has no significance; any fixed number works.

Why scale the features?

CGPA runs 0–10; CodingTestScore runs 0–100; SoftSkillsRating runs 1–5. To gradient descent, the column with the biggest raw numbers appears most important, and the valley becomes a long narrow ravine that the algorithm zig-zags down painfully slowly. `StandardScaler` rewrites every column to have mean 0 and standard deviation 1, turning that ravine into a round bowl.

> **The most important two lines in this step**
>
> Notice: `fit_transform` on training data, but only `transform` on test data.
>
> `fit` means "measure the mean and standard deviation." If you fit on the test set, information from the test set leaks into your preparation, and your reported accuracy becomes an overestimate. The test set must be treated as data you genuinely do not have yet. Getting this backwards is one of the most common and most damaging errors in applied machine learning.

Why add a column of ones?

The model needs a constant b (the intercept) as well as twelve weights. Rather than tracking it separately with its own update rule, we glue a column of 1s onto the front of the data. Then b is just another weight multiplied by 1, and one clean matrix operation handles everything. `np.hstack` is what glues arrays together side by side. After this, X has 13 columns and theta has 13 rows.

Step 3 — Batch Gradient Descent, written by hand

CODE

```
def compute_mse(X, y, theta):
    preds = X @ theta
    errors = preds - y
    return np.mean(errors ** 2)


def batch_gradient_descent(X, y, lr=0.05, n_epochs=500):
    m, n = X.shape
    theta = np.zeros((n, 1))
    loss_history = []

    for epoch in range(n_epochs):
        preds = X @ theta
        errors = preds - y
        gradient = (2 / m) * (X.T @ errors)
        theta -= lr * gradient
        loss_history.append(compute_mse(X, y, theta))

    return theta, loss_history
```

Line by line inside the loop

Line	What is happening
`theta = np.zeros((n, 1))`	Start every weight at zero. This is our arbitrary starting point on the hillside.
`preds = X @ theta`	Predict a salary for all 40,000 training students at once. Because the whole dataset is used in every single step, this is called BATCH gradient descent.
`errors = preds - y`	How wrong is each prediction? Positive means we overestimated.
`X.T @ errors`	X.T flips the matrix on its side. This operation asks, for each of the 13 weights: across all students, how much did this particular feature contribute to the error?
`(2 / m) * ...`	Average it over m students. The 2 comes from differentiating the squared term in the MSE formula.

Line	What is happening
`theta -= lr * gradient`	The actual learning step. Nudge every weight a little way in the direction that reduces error. This one line IS gradient descent.
`loss_history.append(...)`	Record the new error so it can be plotted later. This is how we prove the model is genuinely improving.

One pass through this loop is called an **epoch**. Batch gradient descent performs exactly *one* weight update per epoch, so across 500 epochs theta is adjusted 500 times.

RUNNING IT

```
theta_gd, loss_history_gd = batch_gradient_descent(
    X_train_b, y_train, lr=0.05, n_epochs=500
)

rmse_gd = np.sqrt(compute_mse(X_test_b, y_test, theta_gd))
print(f"[Batch GD]      Test RMSE = {rmse_gd:.4f}")
```

OUTPUT

```
[Batch GD]      Test RMSE = 2.3253
```

Read that as: on unseen students, our from-scratch model's salary prediction is off by about 2.33 lakhs per annum on average. Note that the evaluation uses `X_test_b` and `y_test` — data the training loop never touched.

Step 4 — The scikit-learn cross-check

CODE

```
sk_model = LinearRegression()
sk_model.fit(X_train_s, y_train)

y_pred_sk = sk_model.predict(X_test_s)
rmse_sklearn = np.sqrt(mean_squared_error(y_test, y_pred_sk))

print(f"[sklearn LR]    Test RMSE = {rmse_sklearn:.4f}")
print(f"Difference (GD - sklearn) = {rmse_gd - rmse_sklearn:.6f}")
```

Everything we wrote in twenty lines, scikit-learn does in two. Note it uses `X_train_s` — the version *without* our hand-added column of ones — because sklearn manages its own intercept internally. Feeding it `X_train_b` would give it two intercepts and a subtly wrong answer.

Behind the scenes sklearn does not use gradient descent at all. It solves the problem exactly, in one shot, using linear algebra. So its answer is the *true* best possible straight-line fit, which is precisely what makes it a fair examiner for our iterative approach.

OUTPUT

```
[Batch GD]      Test RMSE = 2.3253
[sklearn LR]    Test RMSE = 2.3253
Difference (GD - sklearn) = 0.000000
```

> **This zero is the result of the experiment**
>
> A difference of 0.000000 means our hand-written gradient descent, starting from all zeros and taking 500 blind downhill steps, arrived at exactly the same answer as an exact algebraic solver. The mathematics is confirmed correct.
>
> If you saw a large difference instead, the cause is almost always one of three things: too few epochs (it has not reached the bottom yet), a learning rate that is too small (steps too tiny) or too large (it is bouncing out of the valley), or forgetting to scale the features.

Step 5 — Mini-batch Gradient Descent

CODE

```
def mini_batch_gradient_descent(X, y, lr=0.05, n_epochs=500,
                                batch_size=256, seed=42):
    m, n = X.shape
    theta = np.zeros((n, 1))
    loss_history = []
    rng = np.random.default_rng(seed)

    for epoch in range(n_epochs):
        indices = rng.permutation(m)
        X_shuffled, y_shuffled = X[indices], y[indices]

        for start in range(0, m, batch_size):
            end = start + batch_size
            X_batch = X_shuffled[start:end]
            y_batch = y_shuffled[start:end]

            preds = X_batch @ theta
            errors = preds - y_batch
            gradient = (2 / X_batch.shape[0]) * (X_batch.T @ errors)
            theta -= lr * gradient

        loss_history.append(compute_mse(X, y, theta))

    return theta, loss_history
```

The one idea that changed

Batch GD looks at all 40,000 students before taking a single step. Mini-batch GD looks at just 256 students, takes a step immediately, then looks at the next 256, and so on. Each individual step is based on less information and so is slightly less accurate — but you take **157 steps per epoch instead of 1**. Many rough steps beat one perfect step.

Why the shuffling matters

`rng.permutation(m)` produces the numbers 0 to 39,999 in random order, and `X[indices]` reorders the data accordingly — at the start of every epoch. Without this, the same 256 students would always travel together in the same batch, and any accidental ordering in the file (say, sorted by college) would bias every update the same way. `seed=42` makes the shuffling reproducible.

One subtle but deliberate choice

The loss is recorded *outside* the inner loop, and it is computed on the *full* training set, not on the last mini-batch. This is what makes the two loss curves comparable: both are "training MSE after N epochs," measured the same way. Recording the last batch's loss instead would produce a wildly jagged, misleading curve.

OUTPUT

```
[Mini-batch GD] Test RMSE = 2.3302
```

Step 6 — Plotting the loss curves

CODE

```
plt.figure(figsize=(8, 5))
plt.plot(loss_history_gd, label="Batch GD")
plt.plot(loss_history_mb, label="Mini-batch GD (batch=256)")
plt.xlabel("Epoch")
plt.ylabel("Training MSE")
plt.title("MSE Loss Curve: Batch GD vs Mini-batch GD")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("loss_curve.png", dpi=150)
```

`plt.plot(list)` draws a line where the x-axis is position in the list (the epoch) and the y-axis is the value (the MSE). `label=` names each line and `plt.legend()` draws the key. `dpi=150` saves at a resolution high enough to paste into a report without blurring.

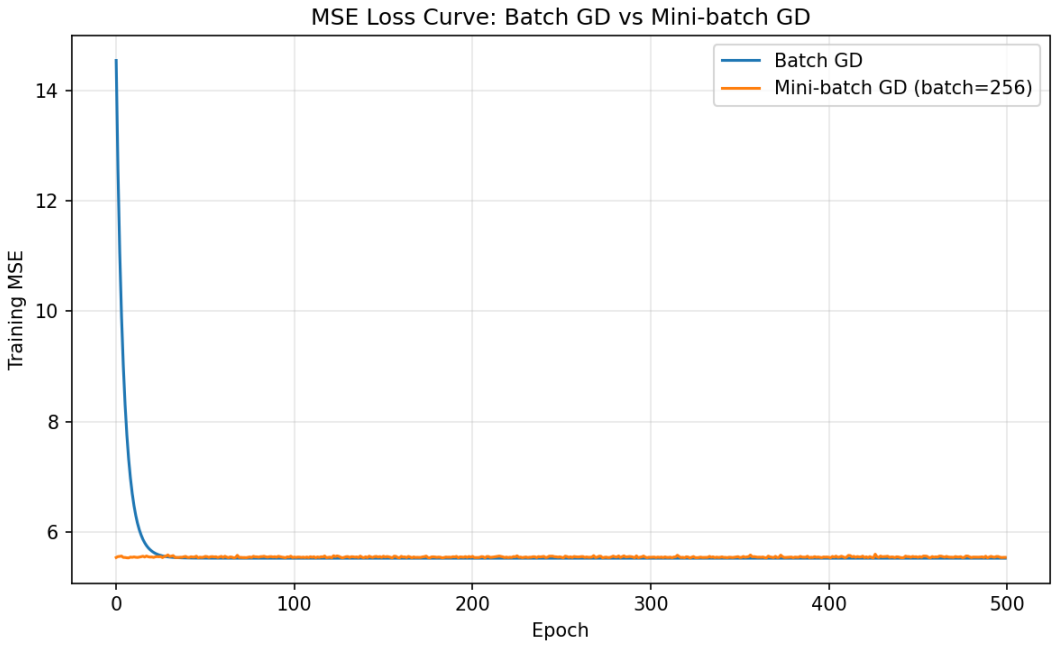

Figure 10: Training MSE against epoch for both methods. Both descend and flatten — the model is learning.

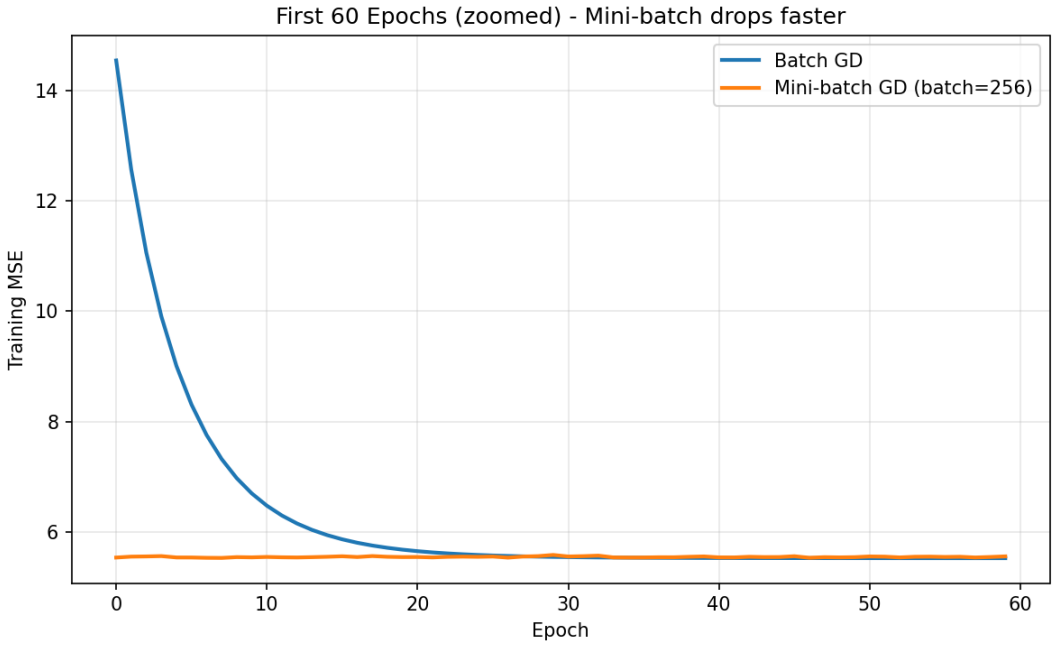

Figure 11: The first 60 epochs enlarged. Mini-batch GD is already near the bottom while batch GD is still descending.

How to read a loss curve — this is a standard viva question

- **It goes down and then flattens.** This is the healthy shape. Flattening means the model has converged; further epochs would achieve nothing.
- **Mini-batch drops far faster at the start.** In our run, batch GD's MSE after one epoch was 14.54, while mini-batch was already at 5.54 — because mini-batch had taken 157 steps in that same epoch instead of one.
- **Mini-batch's line is slightly noisier.** Each step is computed from only 256 students, so it wobbles rather than descending perfectly smoothly. This is expected, and mild noise is even considered useful — it can help escape poor solutions in more complex models.
- **If your curve rises instead of falls**, your learning rate is too large. Try 0.01. If it barely moves at all, it is too small. Try 0.1.

Step 7 — The final comparison table

CODE

```
print("\n=== RMSE Comparison ===")
print(f"{'Method':<20}{'Test RMSE':>12}")
print(f"{'Batch GD':<20}{rmse_gd:>12.4f}")
print(f"{'Mini-batch GD':<20}{rmse_mb:>12.4f}")
print(f"{'sklearn LinearReg':<20}{rmse_sklearn:>12.4f}")
```

`<20` left-aligns text in a 20-character space; `>12.4f` right-aligns a number to 4 decimal places in a 12-character space. That alignment is what makes the columns line up in a plain console with no table library.

OUTPUT

```
=== RMSE Comparison ===
Method                  Test RMSE
Batch GD                   2.3253
Mini-batch GD              2.3302
sklearn LinearReg          2.3253
```

Reading the result

- **Batch GD and sklearn agree to four decimal places.** Our from-scratch implementation found the mathematically optimal solution.
- **Mini-batch GD is very slightly worse (2.3302 vs 2.3253).** Expected. Because it never uses the full dataset for any single update, it hovers around the optimum rather than settling exactly on it. A difference of 0.005 lakhs — about ₹500 a year — is negligible in practice, and it bought a large speed-up in return.
- **The honest caveat.** An RMSE of roughly 2.3 lakhs is not a good salary predictor in absolute terms. That is because, as Program 1 revealed, unplaced students are all recorded at 0 salary. Forcing a single straight line through both the zeros and the real salaries is the wrong model shape. The correct approach would be two stages: first classify placed versus unplaced, then predict salary only for the placed. Stating this limitation in your report will earn you more marks than hiding it.

Batch versus Mini-batch, side by side

	Batch GD	Mini-batch GD
Data used per update	All 40,000 rows	256 rows
Updates per epoch	1	157
Curve shape	Perfectly smooth	Slightly noisy
Speed to converge	Slower (in epochs)	Much faster
Final accuracy	Exactly optimal	Very slightly worse
Memory needed	Whole dataset at once	Only one small batch
Used in practice for	Small datasets	Almost all real deep learning

There is a third variant worth knowing for the viva: **Stochastic Gradient Descent (SGD)** uses a batch size of exactly 1 — it updates after every single student. It is the noisiest and, per update, the fastest. Mini-batch sits deliberately between the two extremes, which is why it is what virtually every neural network in the world is trained with.

Part 4 — Datasets and Download Links

> **About the exact file used in these programs**
>
> The file `placement_predict_50k Dataset.csv` does not appear in a public search of Kaggle or the UCI repository. Its 50,000 rows, 32 columns, and unusual fields such as `IsAnomaly` and eight separate semester SGPA columns strongly suggest it is a synthetic dataset generated for your course. **Ask your lab instructor for it first.**
>
> If you cannot obtain it, the closest public alternatives are listed below, and the appendix contains a script that generates a structurally identical file so both programs will run unchanged.

Closest public alternatives

These are real placement datasets on Kaggle. You will need a free Kaggle account to download. Column names differ, so adjust the FEATURES list accordingly.

- Student Placement Prediction Dataset 2026 — 100,000 students, includes salary, the closest match in size and purpose — https://www.kaggle.com/datasets/sehaj1104/student-placement-prediction-dataset-2026
- Campus Placement Prediction Dataset 2026 — 20,000 records for placement analytics and salary prediction — https://www.kaggle.com/datasets/shambhurajejagadale/campus-placement-prediction-dataset-2026
- Placement Prediction Dataset (Ruchika Kumbhar) — widely used in college projects — https://www.kaggle.com/datasets/ruchikakumbhar/placement-prediction-dataset
- Student Placement Dataset (Sonal Shinde) — large and described as beginner-friendly — https://www.kaggle.com/datasets/sonalshinde123/student-placement-dataset
- Job Placement Dataset — includes 10th, 12th, degree and MBA percentages plus aptitude scores — https://www.kaggle.com/datasets/ahsan81/job-placement-dataset
- College Placement Predictor Dataset — a small, simple CGPA-and-IQ dataset, good for a first attempt — https://www.kaggle.com/datasets/sameerprogrammer/college-placement

General dataset repositories worth bookmarking

- UCI Machine Learning Repository — the classic academic source — https://archive.ics.uci.edu/
- Kaggle Datasets — the largest collection, with notebooks attached — https://www.kaggle.com/datasets
- Google Dataset Search — https://datasetsearch.research.google.com/
- data.gov.in — Indian government open data — https://data.gov.in/
- Hugging Face Datasets — https://huggingface.co/datasets

Downloading from Kaggle directly inside Colab

Rather than downloading to your laptop and re-uploading, you can pull a dataset straight into Colab. Go to Kaggle → your profile → Settings → API → Create New Token. This downloads kaggle.json. Then in Colab:

CODE

```
from google.colab import files
files.upload()          # select your kaggle.json here

!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json

!kaggle datasets download -d sehaj1104/student-placement-prediction-dataset-2026
!unzip -o student-placement-prediction-dataset-2026.zip
```

The ! prefix means "run this as a terminal command, not as Python." Keep kaggle.json private — it is a personal key tied to your account.

Part 5 — How to Run Both Programs Yourself

You do not need to install anything. Google Colab is free, runs in a browser, and already has every library used here. Go to **colab.research.google.com** and sign in with a Google account.

Running Program 1 (the notebook)

12. In Colab choose File → Upload notebook, and select experiment2.ipynb.
13. Click the first cell and press **Shift + Enter** to run it. Use the Choose Files button that appears to upload the CSV.
14. Keep pressing Shift + Enter to work down the notebook **in order**. Order matters enormously — Cell 7 cannot work if Cell 2 has not been run, because df would not exist yet.
15. A number in square brackets appears beside each finished cell. An asterisk [*] means it is still running.
16. The final cell downloads EDA_Report.pdf to your computer.

Running Program 2 (the gradient descent script)

17. Open a new Colab notebook: File → New notebook.
18. Upload the CSV using the same files.upload() cell shown in Part 1.
19. Paste the Program 2 code into a cell and press Shift + Enter. It takes roughly 10–30 seconds — mini-batch GD performs about 78,000 weight updates in total.
20. Check the filename carefully. Program 1 reads `"placement_predict_50k Dataset.csv"` with a space; Program 2 as written reads `"placement_predict_50k_Dataset.csv"` with an underscore. One of the two will fail unless you make them match.

Experiments worth trying (and worth writing up)

Change this	To	And observe
`lr=0.05`	`lr=0.5`	The loss curve may spike upward or produce NaN — the steps overshoot the valley.
`lr=0.05`	`lr=0.001`	The curve descends far too slowly and never converges in 500 epochs.
`batch_size=256`	`batch_size=32`	A noisier curve that converges even faster in terms of epochs.
`batch_size=256`	`batch_size=1`	This is Stochastic Gradient Descent. Very noisy, and slow in wall-clock time.
Remove StandardScaler	—	Convergence becomes dramatically worse. This single experiment proves why scaling matters.
`n_epochs=500`	`n_epochs=50`	Batch GD will not have converged; its RMSE will now differ from sklearn.

Part 6 — Errors You Will Hit, and How to Fix Them

When Python fails it prints a long red block called a **traceback**. Ignore the middle. Read the **last line** — that is the actual problem — and the line number just above it, which tells you where.

Error message	What it means and how to fix it
`FileNotFoundError`	Python cannot find your CSV. Either you did not upload it, or the name does not match exactly. Check spaces, underscores and capital letters. Run !ls in a cell to list what is actually there.
`KeyError: 'Salary Package'`	That column name does not exist in your file. Print df.columns.tolist() and copy the name exactly. Watch for the space in "Salary Package" and for trailing spaces in badly exported CSVs.
`NameError: name 'df' is not defined`	You ran a cell before the cell that creates df. Go back and run the cells in order from the top. In Colab, Runtime → Restart and run all fixes this.
`ModuleNotFoundError: seaborn`	The library is not installed. In Colab, run !pip install seaborn in a cell.
`IndentationError`	Your spacing is inconsistent. Every line inside a for or def must be indented by the same amount. Use four spaces and never mix in tabs.
`ValueError: shapes not aligned`	A matrix multiplication has mismatched dimensions. Print X.shape and theta.shape. Usually you forgot .reshape(-1, 1) on y, or forgot to add the bias column.
`nan` appears in the loss	Your learning rate is too large and the numbers have blown up to infinity. Reduce lr to 0.01 and check that you scaled your features.
`FutureWarning` or `DeprecationWarning`	Not an error at all. Your code worked. The library is telling you this style will change in a future version.
The program runs but a chart is all zeros	The dangerous case — no error at all. This is the Placed_Flag bug from Part 1. Print the intermediate value (df['Placed_Flag'].value_counts()) and check whether it is what you expect.

The debugging habit worth building now

When something looks wrong, do not stare at the code. Print the intermediate values. `.shape` tells you dimensions, `.head()` shows you actual rows, `.dtype` reveals whether a column is numbers or text, and `.value_counts()` shows you what values a column actually contains. Had `print(df['PlacementStatus'].value_counts())` been run before Cell 5, the bug in Program 1 would have been caught in five seconds.

Part 7 — Viva Questions with Model Answers

On Program 1 (EDA)

What is EDA and why do it before modelling?

Exploratory Data Analysis is the process of summarising and visualising a dataset before any model is built. It reveals missing values, outliers, skewed distributions, and relationships between variables. Without it you cannot make informed decisions about cleaning, feature selection, or which model is appropriate.

Why use the median rather than the mean to fill missing values?

The median is the middle value and is unaffected by extreme outliers, whereas a single very large value can drag the mean substantially. Since our boxplots showed outliers in several columns, the median is the safer choice.

What does the missingness heatmap tell you that df.isnull().sum() does not?

The count tells you how many values are missing; the heatmap tells you where. Randomly scattered gaps can safely be filled in. Gaps that cluster in blocks indicate a systematic cause — an entire batch of records failing to load, for instance — and simple imputation would then be misleading.

Why is a high correlation between SGPA columns and CGPA a problem?

It is called multicollinearity. Because CGPA is derived from the semester SGPAs, they carry duplicate information. Including all of them makes the fitted coefficients unstable and hard to interpret, even though overall prediction accuracy may be unaffected.

Your placement-rate chart showed 0% everywhere. Explain.

The Placed_Flag column was built by comparing PlacementStatus against the text "Placed", but PlacementStatus stores the integers 0 and 1. In Python an integer never equals a string, so the condition was always false and every row was assigned 0. Python raised no error because the comparison itself is valid — it simply always returns False. The fix is to compare against the integer 1, or assign the column directly.

On Program 2 (Gradient Descent)

What is gradient descent in one sentence?

An iterative optimisation method that repeatedly adjusts a model's parameters in the direction opposite to the gradient of the loss function, so that the loss decreases step by step until it converges to a minimum.

What is the learning rate, and what happens if it is wrong?

It is the size of each step. Too large and the algorithm overshoots the minimum, causing the loss to oscillate or diverge to infinity. Too small and convergence becomes impractically slow. Here 0.05 was used with 500 epochs.

Why must features be scaled before gradient descent?

Features on different numeric ranges create an elongated, ravine-shaped loss surface that gradient descent traverses very inefficiently, zig-zagging instead of descending directly. Standardising every feature to mean 0 and standard deviation 1 makes the surface approximately circular so a single learning rate suits all directions. Note that scaling is not required for scikit-learn's LinearRegression, since it solves the problem algebraically rather than iteratively.

Why fit the scaler only on training data?

Fitting computes the mean and standard deviation. If those are computed using the test set, information about the test set leaks into training, and the reported test performance becomes an optimistic overestimate of true performance on genuinely unseen data.

Starting from w = 0 and b = 0 with a learning rate of 0.1, work out one iteration on the data (1,3), (2,5), (3,7).

All three predictions are 0, so the errors are −3, −5 and −7. MSE is (9 + 25 + 49)/3 = 27.667. The gradient for w is (2/3)[(−3)(1) + (−5)(2) + (−7)(3)] = (2/3)(−34) = −22.667, and for b it is (2/3)(−15) = −10. Updating: w = 0 − 0.1(−22.667) = 2.267 and b = 0 − 0.1(−10) = 1.0. The loss falls to 0.332 after this single step.

Why does the sign of the errors flip between iteration 1 and iteration 2?

Because the step slightly overshot the minimum. After iteration 1 the line sits marginally above the data rather than below it, so predictions exceed actuals and the errors become positive. The next gradient therefore points the other way and the correction is smaller. This oscillating, shrinking approach to the minimum is normal behaviour, not a fault.

What is the difference between a weight and the bias?

A weight multiplies an input and controls how strongly that feature influences the prediction. The bias is added on regardless of any input and shifts the whole line up or down. In code the bias is usually folded in as an extra weight multiplied by a constant column of ones, so both are updated by the same rule.

Difference between batch, mini-batch and stochastic gradient descent?

Batch uses the whole training set for each update — accurate but slow, one update per epoch. Stochastic uses a single sample per update — very fast per step but extremely noisy. Mini-batch uses a small group, typically 32 to 512 samples, giving most of the speed of stochastic with much of the stability of batch. Mini-batch is the standard choice in practice.

Why add a column of ones to the feature matrix?

To represent the intercept term. By multiplying a constant 1 by an extra weight, the intercept becomes just another parameter inside the same matrix operation, removing the need for separate bias-handling code.

Why does the mini-batch result differ slightly from batch and sklearn?

Mini-batch computes each gradient from a small random subset, so the estimate is noisy. Rather than settling exactly at the minimum, the parameters oscillate in a small region around it. The resulting difference here was about 0.005, which is negligible.

What does an RMSE of 2.33 actually mean?

That on unseen students, predicted salary differs from actual salary by roughly 2.33 lakhs per annum on average. RMSE is expressed in the same units as the target, which is why it is preferred over MSE for reporting.

Why not simply use sklearn and skip writing gradient descent by hand?

Because linear regression has an exact algebraic solution, but almost nothing else does. Neural networks, logistic regression and most modern models can only be trained by gradient descent. Implementing it on a problem where the correct answer is independently known is how you verify your understanding of the algorithm you will rely on everywhere else.

Cross-cutting questions

Why exclude PlacementStatus from the features in Program 2?

It is data leakage. Salary is zero precisely when PlacementStatus is zero, so including it would let the model reproduce the target almost perfectly without learning anything useful about what drives salary. The model would appear excellent in testing and fail completely in real use.

Is this a good salary prediction model?

No, and the EDA explains why. The Salary Package column contains a large spike at zero for unplaced students alongside a continuous distribution for placed students. A single linear model cannot represent both. The correct

design is a two-stage approach: a classifier for placed versus unplaced, then a regressor trained only on placed students.

Appendix — Generating a Stand-In Dataset

If you cannot obtain the original CSV, run this once in Colab. It creates a file with the same 32 columns, the same 50,000 rows, the same missing-value pattern, and the same 0-salary-for-unplaced structure, so both programs run unchanged.

CODE

```
import numpy as np, pandas as pd
rng = np.random.default_rng(7)
n = 50000

cgpa   = np.clip(rng.normal(7.0, 1.1, n), 4, 10)
att    = np.clip(rng.normal(78, 10, n), 40, 100)
intern = rng.poisson(1.0, n)
proj   = rng.poisson(2.0, n)
work   = rng.poisson(1.5, n).astype(float)
cert   = rng.poisson(1.8, n)
pub    = rng.poisson(0.2, n)
apt    = np.clip(rng.normal(60, 14, n), 10, 100)
soft   = np.clip(rng.normal(3.0, 0.8, n), 1, 5)
code_  = np.clip(20 + 6*(cgpa-7) + rng.normal(55,15,n)*0.6, 5, 100)
mock   = np.clip(0.5*code_ + rng.normal(25,10,n), 5, 100)
extra  = rng.integers(0, 2, n)

sgpa = {f"SGPA_Sem{i}": np.clip(cgpa + rng.normal(0,0.35,n), 4, 10).round(2)
        for i in range(1, 9)}

p_placed = 1/(1+np.exp(-(0.9*(cgpa-6.8) + 0.035*(code_-55)
                         + 0.4*intern + 0.3*proj - 1.2)))
placed = (rng.random(n) < p_placed).astype(int)

salary = (2.0 + 0.85*(cgpa-6) + 0.05*(code_-50) + 0.35*intern
          + 0.20*proj + 0.02*(apt-60) + 0.30*soft + 0.015*(att-75)
          + 0.10*cert + 0.25*pub + 0.012*(mock-50) + 0.15*extra
          + rng.normal(0, 0.8, n))
salary = np.round(np.clip(salary, 0, None), 2)
salary[placed == 0] = 0.0

df = pd.DataFrame({
    "StudentID": np.arange(1, n+1),
    "Gender": rng.choice(["Male","Female"], n),
    "City": rng.choice(["Ahmedabad","Mumbai","Kolkata","Jaipur",
                        "Pune","Hyderabad","Chennai","Delhi"], n),
    "CollegeTier": rng.choice(["Tier1","Tier2","Tier3"], n, p=[.25,.5,.25]),
    "Stream": rng.choice(["CS","IT","ECE","EE","Mechanical","Civil"], n),
    "Specialisation": rng.choice(["AI","DataScience","Networking",
                                  "Cybersecurity","Embedded"], n),
    "Hostel": rng.choice(["Yes","No"], n),
    "HistoryOfBacklogs": rng.choice(["Yes","No"], n, p=[.25,.75]),
    **sgpa,
    "CGPA": cgpa.round(2), "AttendancePercent": att.round(1),
    "Internships": intern, "Projects": proj, "Workshops": work,
    "Certifications": cert, "Publications": pub,
    "AptitudeTestScore": apt.round(1), "SoftSkillsRating": soft.round(1),
    "CodingTestScore": code_.round(1), "MockInterviewScore": mock.round(1),
    "ExtraCurricular": extra,
    "CGPA_Tier": pd.cut(cgpa, [0,6.5,8,10],
                        labels=["Low","Mid","High"]).astype(str),
    "PlacementStatus": placed,
    "IsAnomaly": (rng.random(n) < 0.02).astype(int),
    "Salary Package": salary,
})

# recreate the missing-value pattern seen in the original file
for col, frac in [("Workshops",.090), ("AptitudeTestScore",.081),
                  ("SoftSkillsRating",.071), ("CodingTestScore",.060),
                  ("MockInterviewScore",.099)]:
    idx = rng.choice(n, int(frac*n), replace=False)
    df.loc[idx, col] = np.nan
```

```
df.to_csv("placement_predict_50k Dataset.csv", index=False)
print(df.shape)
```

Declare it if you use it

If you submit work based on this generated file, say so plainly in your report: "the original dataset was unavailable, so a synthetic dataset with identical structure was generated." Examiners respect that far more than an unexplained discrepancy in your numbers. It also means your results cannot be compared directly with classmates using the real file.